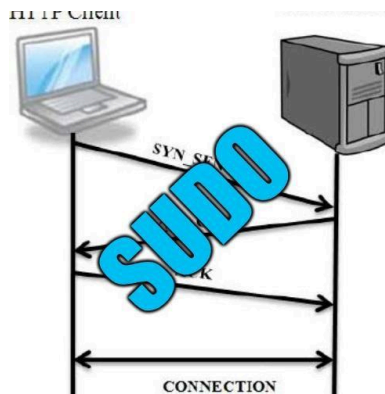


AUDITORÍA - SISTEMA WEB

Grado en Ingeniería Informática de Gestión y Sistemas de Información
Sistemas de Gestión de Seguridad de Sistemas de Información



Componentes:

Antía Arean Rodríguez

Pablo Fernández González

Jannatul Ferdous Hossain Begum

Ziyan Jiang

Aitana Niño Bedoya

Fecha de entrega: 30/10/2025

ÍNDICE

1. INTRODUCCIÓN.....	2
2. RESULTADOS DE LA AUDITORÍA CON ZAP.....	3
3. VULNERABILIDADES.....	4
3.1. Inyección SQL - MySQL.....	4
3.2. Cabecera Content Security Policy (CSP) no configurada.....	5
3.3. Divulgación de error de aplicación.....	7
3.4. Divulgación de información mediante un campo de encabezado de respuesta HTTP.....	9
3.5. Falta de cabecera Anti-Clickjacking.....	9
3.6. Cookie sin flag HttpOnly.....	10
3.7. Cookie sin el atributo SameSite.....	10
3.8. El servidor divulga información mediante un campo(s) de encabezado de respuesta HTTP ""X-Powered-By""	11
3.9. El servidor filtra información de versión a través del campo "Server" del encabezado de respuesta HTTP.....	12
3.10. Falta encabezado X-Content-Type-Options.....	12
4. BIBLIOGRAFÍA.....	13

1. INTRODUCCIÓN

Para encontrar vulnerabilidades en nuestro sistema web, se ha utilizado OWASP Zed Attack Proxy (ZAP). Esta es una herramienta desarrollada por OWASP (Open Web Application Security Project), un proyecto abierto de seguridad en aplicaciones web.

ZAP funciona como un proxy “in-the-middle”, es decir, intercepta e inspecciona los mensajes enviados entre el navegador y la aplicación, alterando su contenido si es necesario.



Esta es la lista de los riesgos de seguridad más críticos según OWASP (última versión oficial de 2021):

A1 - Control de acceso roto
A2 - Fallos criptográficos
A3 - Inyección
A4 - Diseño inseguro
A5 - Configuración incorrecta de seguridad
A6 - Componentes vulnerables y obsoletos
A7 - Fallo de identificación y autenticación
A8 - Fallo de la integridad del software y datos
A9 - Fallo de registro y monitoreo de seguridad
A10 - Falsificación de solicitudes del lado del servidor

Para instalar ZAP hemos seguido los siguientes pasos:

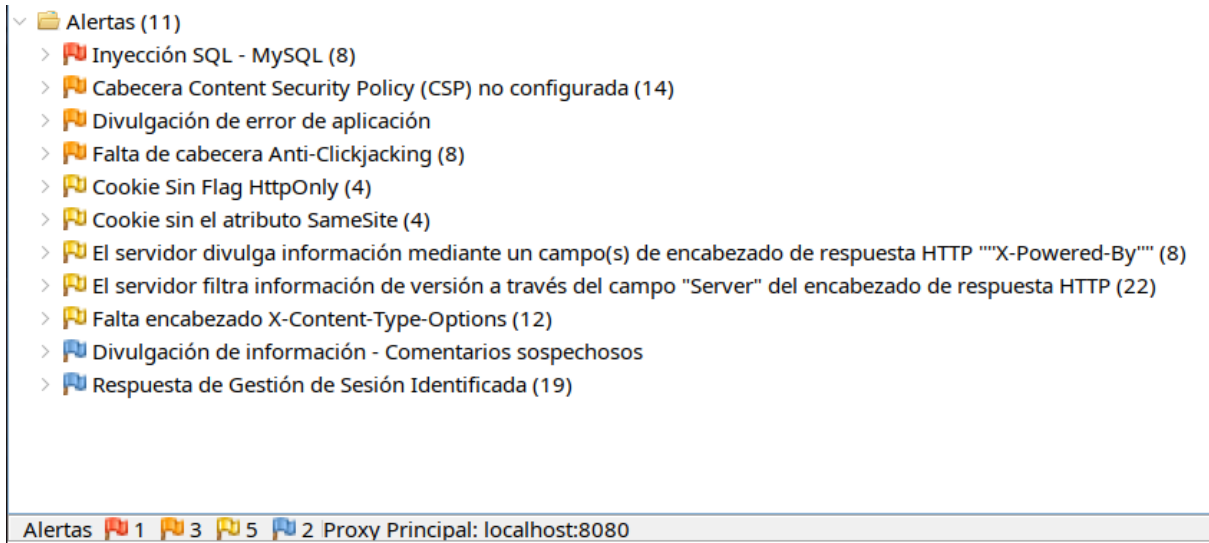
1. Descargamos de <https://www.zaproxy.org/download/> el “Linux Package”. Que cuenta con el ZAP 2.16 package”.
2. Zap necesita java 17 o superior para funcionar, entonces instalamos openjdk, un paquete apt que dará a nuestra máquina compatible con java
3. Descomprimos el archivo comprimido que nos hemos descargado de la página de ZAP y ejecutamos el script de bash “zap.sh” que contiene
4. No hemos tocado los puertos que utiliza ZAP, escaneamos desde el por defecto, el 8080
5. Le damos al botón de escanear rápido y en el campo de la URL ponemos la de nuestro sistema (Localhost:81)

2. RESULTADOS DE LA AUDITORÍA CON ZAP

El escaneo con OWASP ZAP identificó 11 vulnerabilidades de seguridad en nuestro sistema web, distribuidas en diferentes niveles de criticidad. En la que podemos observar una vulnerabilidad de alto riesgo, 3 vulnerabilidades de medio riesgo, 5 vulnerabilidades de bajo riesgo y 2 vulnerabilidades informativas.

El análisis reveló que nuestras principales debilidades se concentran en la configuración de seguridad insuficiente, inyección y diseño inseguro.

En la siguiente captura podemos ver las 11 alertas detectadas:



The screenshot shows the 'Alerts' section of the OWASP ZAP interface. It lists 11 detected alerts, each with a severity icon (red for high, orange for medium, yellow for low, and blue for informational) and a count in parentheses. The alerts are:

- > Inyección SQL - MySQL (8)
- > Cabecera Content Security Policy (CSP) no configurada (14)
- > Divulgación de error de aplicación
- > Falta de cabecera Anti-Clickjacking (8)
- > Cookie Sin Flag HttpOnly (4)
- > Cookie sin el atributo SameSite (4)
- > El servidor divulga información mediante un campo(s) de encabezado de respuesta HTTP ""X-Powered-By"" (8)
- > El servidor filtra información de versión a través del campo "Server" del encabezado de respuesta HTTP (22)
- > Falta encabezado X-Content-Type-Options (12)
- > Divulgación de información - Comentarios sospechosos
- > Respuesta de Gestión de Sesión Identificada (19)

At the bottom, a summary bar shows: 'Alertas' followed by icons and counts: 1 high risk, 3 medium risk, 5 low risk, and 2 informational. It also states 'Proxy Principal: localhost:8080'.

3. VULNERABILIDADES

3.1. Inyección SQL - MySQL

Hablamos de ‘**Inyección de SQL**’ cuando un hacker inyecta código SQL malicioso en nuestro sistema y consigue que alcance la base de datos. Estos solo son viables cuando la base de datos carece de mecanismos de ‘saneamiento de entrada’ que impidan que el código de los usuarios se pueda colar por los resquicios de nuestro sistema y funcionar como código ejecutable en el servidor

En consecuencia, existen varios tipos de inyección sql dependiendo del método que el atacante utilice para enmascarar su ‘malware’; **inyección mediante modificación de cookies, inyección mediante variables del servidor, inyecciones mediante herramientas de hackeo automáticas...** Pero en esta auditoría discutiremos solamente a la que hemos detectado que nuestro sistema es especialmente vulnerable, **inyección de sql mediante introducción de datos del usuario**

Las ‘**Inyecciones de sql mediante introducción de datos del usuario**’ se produce cuando el atacante concatena su código en las variables especificadas por el usuario y este se ejecuta junto con el resto de la consulta SQL. Es el método de inyección de SQL más común. A continuación se incluye un ejemplo de código susceptible a este tipo de ataque, por la forma en que se insertan valores en la consulta de sql:

```
$valor = $_GET['valor'];  
$SQL = "SELECT * FROM tabla WHERE id = $valor";  
$result = mysqli_query($conn, $SQL);
```

También se adjunta un ejemplo de este mismo error en nuestro código:

```
include 'bdcon.php';  
  
$orden = $_GET['orden'] ?? 'modelo';  
  
$sql = "  
SELECT  
    c.modelo,  
    c.marca,  
    c.kilometraje,  
    c.color,  
    u.username AS vendedor,  
    c.matricula,  
    c.precio  
  
FROM coches c  
LEFT JOIN usuarios u ON u.id = c.id_propietario  
WHERE c.en_venta = '1'  
ORDER BY $orden  
";
```

Implementar parámetros de esta manera en nuestras consultas de SQL permitirá a atacantes leer, borrar o incluso alterar la información. Esto puede ser especialmente dañino en un sistema web como el nuestro, donde la interacción con la base de datos es clave para el funcionamiento del proceso de la compra. De no ser corregido, permitiría, por ejemplo, a un usuario malicioso, darse cantidades de dinero infinitas, o quitar dinero a otros usuarios.

3.2. Cabecera Content Security Policy (CSP) no configurada

Content Security Policy (CSP) es una capa de seguridad adicional que ayuda a prevenir y mitigar algunos tipos de ataque. Funciona como una lista blanca de fuentes permitidas, previniendo ataques como: Cross-Site Scripting (XSS), inyección de contenido malicioso, Clickjacking y data exfiltration. Estos ataques se utilizan para todo, desde el robo de datos hasta la desfiguración del sitio o la distribución de malware.

Tenemos clasificado esta vulnerabilidad como gravedad media y confianza alta y se corresponde específicamente a la configuración de seguridad insuficiente (A05:2021).

Análisis de las causas en nuestra página web:

- Nuestros archivos como index.php, inicio.php, catalogo.php carecen de la cabecera CSP.

```
<?php
// Cabecera CSP en PHP
header("Content-Security-Policy:                default-src
'self'");
?>
```

- Carga de recursos sin restricciones

Los archivos afectados son todos aquellos archivos PHP que generan respuestas HTTP, como index.php, inicio.php, catalogo.php, show_user.php, etc.

Ejemplos específicos en nuestro sistema:

```
<link rel="stylesheet" href="/css/style.css">
<link rel="shortcut icon" href="media/icon.svg" />
<script src="js/validarDatos.js"></script>
```

index.php

```
<script src="../../js/jquery-3.5.1.min.js"></script>
```

catalogo.php

- Uso de JavaScript Inline y Estilos Inline

El código JavaScript se escribe directamente dentro de los atributos HTML de los elementos en lugar de estar en archivos .js externos o en secciones <script> separadas.

Problema: innerHTML inserta HTML directamente, lo que puede permitir inyección de código si no se sanitiza adecuadamente.

```
<script>
//para cargar contenido
function mostrar(tipo) {
  const div = document.getElementById('contenido');
  if(tipo === 'login'){
    div.innerHTML = `
      <h3>Iniciar sesión</h3>
      <form action="/api/login.php" method="POST">
        <label>Usuario:</label><br><input type="text" name="usuario" id="usuario" required><br>
        <label>Contraseña:</label><br><input type="password" name="password" id="password" required><br><br>
        <input type="submit" value="Entrar">
      </form>
    `;
  } else if(tipo === 'registro'){ //VALIDANDO FORMATO
    div.innerHTML = `
      <h3>Registrarse</h3>
      <form action="/api/register.php" method="POST" onsubmit="return validarDni()">
        Nombre: <br><input type="text" name="nombre" required><br>
        Apellidos: <br><input type="text" name="apellidos" required><br>
        DNI (formato 11111111-X): <br><input type="text" name="dni" id="dni" maxlength="10" pattern="^\d{8}-[A-Z]$" required
        title="Debe tener 8 números, un guion y una letra mayúscula, como 12345678-Z" placeholder="11111111-X"><br>
        Tlf: <br><input type="text" name="telefono" pattern="^\d{9} $" maxlength="9" required placeholder="123456789"><br>
        Fecha nacimiento: <br><input type="date" name="fecha_nacimiento" required><br>
        Email: <br><input type="email" name="email" required><br>
        Username: <br><input type="text" name="username" required><br>
        Contraseña: <br><input type="password" name="password" required><br><br>
        <input type="submit" value="Registrarse">
      </form>
    `;
  }
}
</script>
```

index.php

Estilo inline: las reglas CSS se aplican directamente a elementos HTML mediante el atributo style en lugar de estar en .css externos.

```
// Contraseña incorrecta
echo "<h3>Error al iniciar sesión, contraseña incorrecta.</h3>";
echo "<a href='../index.php' style='display: inline-block; padding: 10px 15px;
background-color: #007bff; color: white; text-decoration: none; border-radius: 4px;'>Volver al inicio</a>";
echo "</div>";
```

inicio.php

El hecho de que ZAP haya detectado CSP no configurada significa que nuestra página web carece por completo de esta importante capa de seguridad. Los navegadores no reciben instrucciones sobre qué recursos están permitidos cargar, dejando el sistema web vulnerable a varios tipos de ataques.

3.3. Divulgación de error de aplicación

La divulgación de error de aplicación ocurre cuando una aplicación muestra mensajes de error o advertencia que revelan información sensible del sistema, por ejemplo rutas y nombres de archivos, trazas de ejecución (stack traces), detalles de la base de datos, versiones de software o estructura de ficheros. Esta información puede ayudar a un atacante a diseñar ataques más dirigidos contra el sistema web.

Este riesgo de gravedad media corresponde a la categoría de configuración de seguridad insuficiente.

En nuestra página web, se muestra mensajes de error detallados en login.php:

```
// Verificar contraseña
if ($password === $row['password']) {
    // Login correcto - Guardar datos en sesión
    $_SESSION['user_id'] = $row['id'];
    $_SESSION['usuario'] = $row['username'];
    $_SESSION['nombre'] = $row['nombre'];
    $_SESSION['es_admin'] = $row['es_admin'];

    // Redirigir a pagina principal
    header("Location: inicio.php");
    exit();
} else {
    // Contraseña incorrecta
    echo "<h3>Error al iniciar sesión, contraseña incorrecta.</h3>";
    echo "<a href='../index.php' style='display: inline-block; padding: 5px 10px;'>Volver</a>";
    echo "</div>";
    exit();
}
} else {
    // Usuario no encontrado
    echo "<h3>Error al iniciar sesión, usuario no encontrado.</h3>";
    echo "<a href='../index.php' style='display: inline-block; padding: 5px 10px;'>Volver</a>";
    echo "</div>";
}
}
```

El código presenta una vulnerabilidad de enumeración de usuarios debido a que los mensajes de error son demasiado específicos. Cuando un usuario introduce credenciales incorrectas, la aplicación responde con mensajes distintos según el caso, esto es, cuando se introduce un usuario existente pero una contraseña incorrecta, el mensaje será “*contraseña incorrecta*”, mientras que si el usuario no existe, el mensaje será “*usuario no encontrado*”. Esto es un problema crucial, ya que permite a un atacante diferenciar entre usuarios válidos e inválidos, facilitando ataques de fuerza bruta dirigidos.

Otra divulgación de error es la exposición de información de la base de datos. El problema está en que múltiples archivos de nuestra aplicación cuando ocurre un error en una consulta a la base de datos, se expone el mensaje de error original de MySQL directamente al usuario. A partir de dichos errores, un atacante podría deducir la información crítica asociada a la estructura de las tablas y relaciones de la base de datos.

Ejemplos en nuestro código:

```
if ($conn->query($sql) === TRUE) {
    // Guardar datos de sesión
    $_SESSION['user_id'] = $conn->insert_id;
    $_SESSION['usuario'] = $username;
    $_SESSION['nombre'] = $nombre;
    $_SESSION['es_admin'] = 0;

    // Redirigir
    header("Location: inicio.php");
    exit();
} else {
    echo "Error: " . $conn->error;
}
?>
```

register.php

```
if (mysqli_query($conn, $sql_update)) {
    echo "<div class='alert alert-success'>Datos actualizados correctamente.</div>";
} else {
    echo "<div class='alert alert-error'>Error al actualizar: " . mysqli_error($conn) . "</div>";
}
```

show_user.php

```
// Ejecutar la consulta
if (mysqli_query($conn, $sql)) {
    header("Location: mis_coche.php?id=$coche_id"); // Redirige a mis_coche.php con un parámetro de éxito
    exit();
} else { // Si falla, crea un mensaje de error
    $mensaje = "<div class='alert alert-error'>Error al actualizar el coche: " . mysqli_error($conn) . "</div>";
}
```

modificar_coche.php

3.4. Divulgación de información mediante un campo de encabezado de respuesta HTTP

El servidor de la web está divulgando información mediante uno o más encabezados de respuesta HTTP “X-Powered-By”. Estos son generados por defecto en Apache y PHP.

```
"X-Powered-By: PHP/7.2.2"
```

Este error puede causar que un atacante identifique los componentes de los que depende la web, y realizar ataques específicos. En este caso se expone el backend de la web (PHP) y su versión (7.2.2).

3.5. Falta de cabecera Anti-Clickjacking

Este encabezado permite al sitio web controlar cómo debe ser incrustado en un marco o iframe por otros sitios web.

Análisis de las causas en nuestra página web:

- Ausencia total de cabeceras Anti-Clickjacking: nuestros archivos como login.php, register.php carecen de la cabecera Anti-Clickjacking.

Ejemplos:

```
<?php
session_start();

// Conexion a la base de datos
include 'bdcon.php';
```

login.php

```
<?php
session_start();
// Conexión a la base de datos
include 'bdcon.php'; // Este archivo debe definir: $conn
```

register.php

- Uso del DENY y Content-Security-Policy:

Deny: Este valor indica al navegador que no muestre el sitio web en ningún marco.

Content-Security-Policy: reemplazo más moderno (seguro y flexible)

```
<?php
// Cabecera Anti-Clickjacking en PHP
header("X-Frame-Options: DENY")
header("Content-Security-Policy:      frame-ancestors
'none' )
?>
```

3.6. Cookie sin flag HttpOnly

Se ha establecido una cookie sin la bandera HttpOnly, lo que significa que se puede acceder a la cookie por JavaScript.

Análisis de las causas en nuestra página web:

➔ Ausencia total de cookie sin flag HttpOnly: nuestros archivos como login.php, register.php entre otras carecen de una cookie sin flag.

- Uso del session_set_cookie_params

Session_set_cookie_params: Sirve para que no se pueda robar las cookies de sesión.

```
<?php
session_set_cookie_params([
    'httpOnly'=> true
])
?>
```

3.7. Cookie sin el atributo SameSite

Se ha establecido una cookie sin el atributo SameSite, esto quiere decir que el navegador hará que la cookie envíe solicitudes entre sitios. Esto es un riesgo de seguridad, porque aumenta la posibilidad de ataques CSRF (Cross-Site Request Forgery) o de seguimiento no deseado del usuario a través de diferentes dominios.

- Análisis en nuestra página web:

En los scripts actuales (login.php, register.php y otros), las cookies de sesión se crean sin especificar el parámetro SameSite.

Esto hace que el navegador pueda enviar la cookie incluso si la solicitud proviene de otro dominio (por ejemplo, desde un enlace malicioso).

3.8. El servidor divulga información mediante un campo(s) de encabezado de respuesta HTTP `'''X-Powered-By'''`

El servidor está revelando información sensible mediante el encabezado X-Powered-By. Se recomienda ocultarlo para reducir la superficie de ataque.

Este encabezado revela la tecnología y versión utilizadas en el backend (por ejemplo, PHP, ASP.NET, etc.).

Un atacante podría usar esta información para buscar vulnerabilidades específicas de esa versión.

- Análisis en nuestra página web:

El encabezado X-Powered-By es enviado automáticamente por PHP cuando `expose_php` está habilitado en la configuración del servidor (`php.ini`).

Esto ocurre en todos los archivos PHP servidos por el servidor.

- Solución propuesta:

```
expose_php = off
```

Además en la raíz del proyecto se pondría:

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
  <system.webServer>
    <httpProtocol>
      <customHeaders>
        <remove name="X-Powered-By" />
      </customHeaders>
    </httpProtocol>
  </system.webServer>
</configuration>
```

Por último habría que reiniciar Visual Studio y verificarlo en el PowerShell.

3.9. El servidor filtra información de versión a través del campo "Server" del encabezado de respuesta HTTP

El servidor web revela información de versión mediante el encabezado Server en las respuestas HTTP. Esto expone detalles sobre la infraestructura que pueden ser utilizados durante la fase de reconocimiento de un ataque. Se recomienda configurar el servidor para ocultar o minimizar la información expuesta en dicho encabezado.

3.10. Falta encabezado X-Content-Type-Options

El encabezado X-Content-Type-Options no está presente en las respuestas HTTP.

Este encabezado evita que el navegador interprete archivos de un tipo distinto al declarado, protegiendo frente a ataques de MIME sniffing.

- Análisis en nuestra página web:

El servidor no incluye este encabezado por defecto, lo que permite que el navegador intente “adivinar” el tipo de contenido de un archivo.

Esto podría usarse para ejecutar código malicioso si un archivo subido se sirve con un tipo incorrecto.

4. BIBLIOGRAFÍA

- Belcic, I. (2020, 22 septiembre). *¿Qué es la inyección de SQL y cómo funciona?*
<https://www.avast.com/es-es/c-sql-injection#:~:text=Inyecci%C3%B3n%20de%20SQL%20mediante%20la%20modificaci%C3%B3n%20de%20cookies,a%20la%20base%20de%20datos.>
- Cloudflare. (s. f.). *¿Qué es la inyección de código SQL?* Recuperado 30 de octubre de 2025, de <https://www.cloudflare.com/es-es/learning/security/threats/sql-injection>
- Egaña Aranguren, M. (2024, 23 Octubre). *Seguridad en Sistemas Web*. GitHub. Recuperado 30 de octubre de 2025, de https://github.com/mikel-egana-aranguren/EHU-SGSSI-01/blob/main/Seguridad_web/index.pdf
- MDN contributors. (s.f.) - *Política de seguridad del contenido (CSP)*. MDN. Recuperado 30 de octubre de 2025, de <https://developer.mozilla.org/es/docs/Web/HTTP/Guides/CSP>
- Microsoft. (2025, 30 junio). *Inyección de código de SQL*. Microsoft Learn.
<https://learn.microsoft.com/es-es/sql/relational-databases/security/sql-injection?view=sql-server-ver17>
- Roviralta Puente, J. M. (2021, 26 octubre). *Ataques de inyección SQL, una amenaza para tu web*. INCIBE.
<https://www.incibe.es/empresas/blog/ataques-inyeccion-sql-amenaza-tu-web#:~:text=%C2%BFC%C3%B3mo%20se%20realizan%20estos%20ataques,en%20la%20base%20de%20datos.>
- Serrano Garzón, L. M. (s.f.) - *Test de penetración a la aplicación web Badstore utilizando una herramienta de análisis dinámico*. Scribd. Recuperado 30 de octubre de 2025, de <https://es.scribd.com/document/639736532/OWASP>
- StackHawk. (s.f.) - *Encabezado de opciones de marco X no configurado*. Stackhawk
<https://docs.stackhawk.com/vulnerabilities/10020/#:~:text=About-,The%20vulnerability%20%E2%80%9CX%2DFrame%2DOptions%20Header%20Not%20Set%E2%80%9D,disguised%20as%20a%20legitimate%20website.>

Toledo, F. (2017, 20 noviembre). *Introducción al Testing de Seguridad con OWASP ZAP*.
<https://federico-toledo.com/introduccion-al-testing-de-seguridad/>

ZAP. – documentation. (s.f.) ZAP. Recuperado 30 de octubre de 2025, de
<https://www.zaproxy.org/docs/>

ZAP. – Getting started. (s.f.) ZAP. Recuperado 30 de octubre de 2025, de
<https://www.zaproxy.org/getting-started/>

ZAP. - ZAPping the OWASP Top 10. (2021) ZAP.
<https://www.zaproxy.org/docs/guides/zapping-the-top-10-2021/>